

CS 423

Operating System Design: Locks (continued)

10/08

Ram Alagappan

Slide ack: Prof. Shivaram Venkataraman (Wisconsin)

Logistics

MPI grades released

Talk to our TAs if you need regrades

Some minor changes in schedule – semaphores on 10/15 (before midterm).

AGENDA / LEARNING OUTCOMES

So far:

- Locks: how to use and how to implement

- HW atomic instructions to implement locks (TAS, CAS)

Today:

- How to make these locks fair?

- Spin if not able to acquire locks

- Do something better than spinning (queue locks)

- Start Condition variables

RECAP

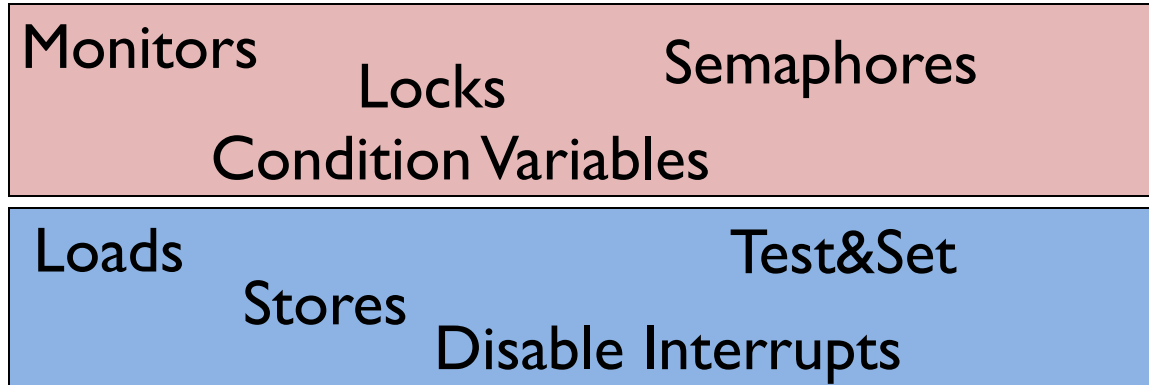
Synchronization

Build higher-level synchronization primitives in OS

Operations that ensure correct ordering of instructions across threads

Use help from hardware

Motivation: Build them once and get them right



LOCK IMPLEMENTATION GOALS

Correctness

- *Mutual exclusion (safety)*
Only one thread in critical section at a time
- *Progress (liveness)*
If several simultaneous requests, must allow one to proceed

Fairness: does each thread have a fair shot at acquiring? Does anybody starve?

Performance: CPU is not used unnecessarily (spinning)

Implementing Locks

Approaches

- Disable interrupts - not good: doesn't work on multi processors, cannot perform IO, malicious thread can take CPU for arbitrarily long
- Load and stores of words - not good: we don't build locks this way (more theoretical)
- Using atomic hardware instructions (e.g., test and set)

LOCK Implementation with TAS

```
1  typedef struct __lock_t {
2      int flag;
3  } lock_t;
4
5  void init(lock_t *lock) {
6      // 0: lock is available, 1: lock is held
7      lock->flag = 0;
8  }
9
10 void lock(lock_t *lock) {
11     while (TestAndSet(&lock->flag, 1) == 1)
12         ; // spin-wait (do nothing)
13 }
14
15 void unlock(lock_t *lock) {
16     lock->flag = 0;
17 }
```

Lock Using CAS

```
int CompareAndSwap(int *addr, int expected, int new) {  
    int actual = *addr;  
    if (actual == expected)  
        *addr = new;  
    return actual;  
}
```

```
void acquire(lock_t *lock) {  
    while(CompareAndSwap(&lock->flag, 0 , 1 ) == 1) ;  
    // spin-wait (do nothing)  
}
```

END RECAP

Chat for a minute...

```
int xchg(int *addr, int newval) {  
    int old = *addr;  
    *addr = newval;  
    return old;  
}
```

```
int CompareAndSwap(int *addr,  
int expected, int new) {  
    int actual = *addr;  
    if (actual == expected)  
        *addr = new;  
    return actual;  
}
```

```
a = 1  
int b = xchg(&a, 2)  
int c = CAS(&b, 2, 3)  
int d = CAS(&b, 1, 3)
```

Final values?

LOCK IMPLEMENTATION GOALS

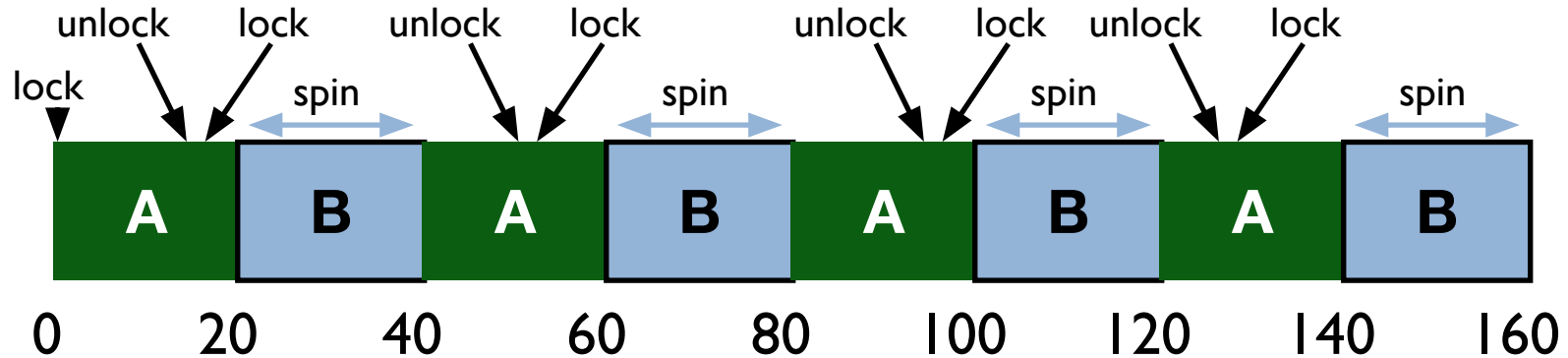
Correctness

- *Mutual exclusion*
Only one thread in critical section at a time
- *Progress* (deadlock-free)
If several simultaneous requests, must allow one to proceed

Fairness: does each thread have a fair shot at acquiring? Does anybody starve?

Performance: CPU is not used unnecessarily (spinning)

BASIC SPINLOCKS ARE UNFAIR



Scheduler is unaware of locks/unlocks!
B is unlucky - never is able to acquire lock

FAIRNESS: TICKET LOCKS

Idea: reserve each thread's turn to use a lock.

Each thread spins until their turn.

Use new atomic primitive, fetch-and-add

```
int FetchAndAdd(int *ptr) {  
    int old = *ptr;  
    *ptr = old + 1;  
    return old;  
}
```

Acquire: Grab ticket; Spin while not thread's ticket != turn

Release: Advance to next turn

TICKET LOCK IMPLEMENTATION

```
typedef struct __lock_t {  
    int ticket;  
    int turn;  
}
```

```
void lock_init(lock_t *lock) {  
    lock->ticket = 0;  
    lock->turn = 0;  
}
```

```
void acquire(lock_t *lock) {  
    int myturn = FAA(&lock->ticket);  
    // spin  
    while (lock->turn != myturn);  
}
```

```
void release(lock_t *lock) {  
    lock->turn = lock->turn + 1  
}
```

Example

A.lock myturn = 0, ticket = 1

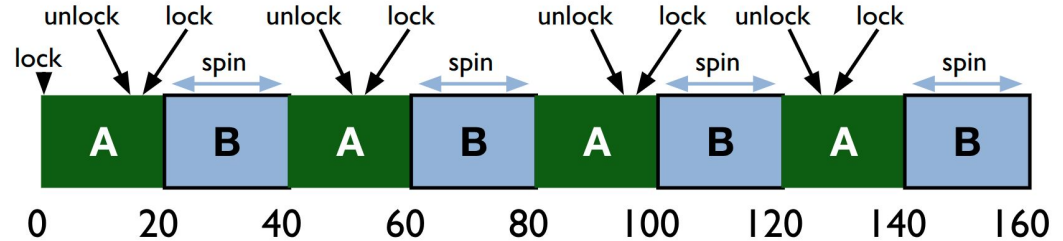
A.unlock turn =1

A. lock myturn = 1, ticket = 2

B.lock myturn =2, ticket = 3

A.unlock turn = 2

A.lock myturn=3, ticket = 4 // A cannot acquire lock now!



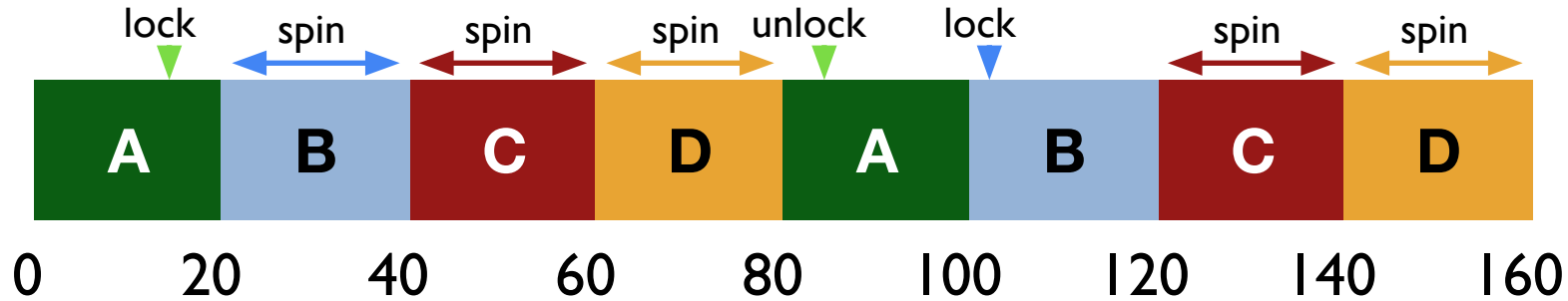
This is how ticket locks enable fairness (different from the example shown in slide 13)

SPINLOCK PERFORMANCE

Slow when...

- locks held a long time
- disadvantage: spinning is wasteful
- especially bad with one CPU

CPU SCHEDULER IS IGNORANT



CPU scheduler may run **B, C, D** instead of **A**
even though **B, C, D** are waiting for **A**

Solution: Yield (voluntarily deschedule yourself)

TICKET LOCK WITH YIELD

```
typedef struct __lock_t {  
    int ticket;  
    int turn;  
}
```

```
void lock_init(lock_t *lock) {  
    lock->ticket = 0;  
    lock->turn = 0;  
}
```

```
void acquire(lock_t *lock) {  
    int myturn = FAA(&lock->ticket);  
    while (lock->turn != myturn)  
        yield();  
}
```

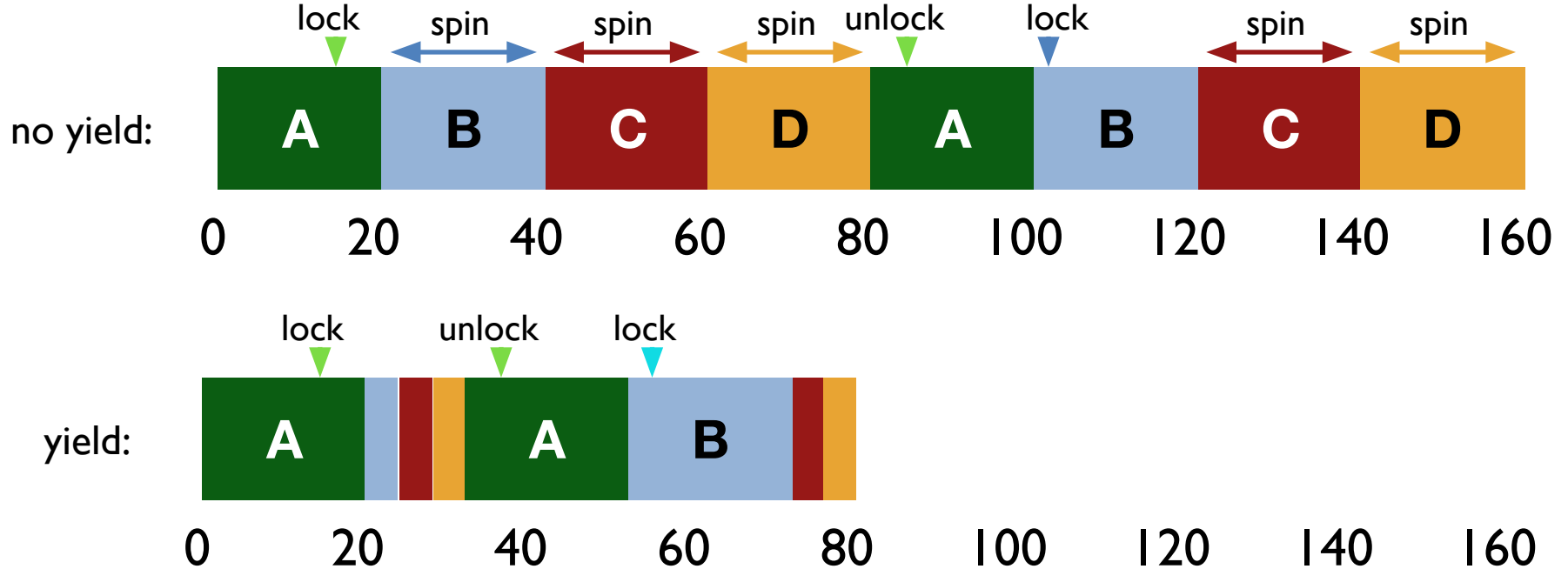
```
void release(lock_t *lock) {  
    lock->turn = lock->turn + 1  
}
```

`yield()` voluntarily relinquishes the CPU for remainder of time slice, but process remains READY

TAS Lock with Yield

```
void init() {  
    flag = 0;  
}  
  
void lock() {  
    while (TestAndSet(&flag, 1) == 1)  
        yield(); // give up the CPU  
}  
  
void unlock() {  
    flag = 0;  
}
```

Yield Instead of Spin



YIELD VS SPIN

Waste of CPU cycles?

Without yield: $O(\text{threads} * \text{time_slice})$

With yield: $O(\text{threads} * \text{context_switch})$

While yield is better than spinning, with high contention, it can also be bad

Problem: the thread is still in READY state (and so can be scheduled on CPU)

Still context switches happen between all READY threads, which is inefficient

Next improvement: Block and put thread on waiting queue

Lock Implementation: Block when Waiting

Remove waiting threads from scheduler runnable queue

Scheduler runs any thread that is **runnable**

Support in Solaris OS: `park()`, `unpark()`

Support in Linux: `futex` (fast userspace mutex)

Block when Waiting

```
typedef struct {  
    bool lock = false;  
    bool guard = false;  
    queue_t q;  
} LockT;
```

```
void acquire(LockT *l) {  
    while (XCHG(&l->guard, true));  
    if (l->lock) {  
        qadd(l->q, tid);  
        l->guard = false;  
        park();      // blocked  
    } else {  
        l->lock = true;  
        l->guard = false;  
    }  
}
```

```
void release(LockT *l) {  
    while (XCHG(&l->guard, true));  
    if (qempty(l->q)) l->lock=false;  
    else unpark(qremove(l->q));  
    l->guard = false;  
}
```

Block when Waiting

(a) Why is **guard** used? What does it protect?

(b) Why okay to **spin** on guard?

(c) In release(), why not set lock=false when unpark? What if we set lock=false when unparking?

```
void acquire(LockT *l) {
    while (XCHG(&l->guard, true));
    if (l->lock) {
        qadd(l->q, tid);
        l->guard = false;
        park();      // blocked
    } else {
        l->lock = true;
        l->guard = false;
    }
}

void release(LockT *l) {
    while (XCHG(&l->guard, true));
    if (qempty(l->q)) l->lock=false;
    else unpark(qremove(l->q));
    l->guard = false;
}
```

Block when Waiting

(d) What if order of guard=false and park() is changed?

(e) Is there a case where a thread may indefinitely sleep?

```
void acquire(LockT *l) {
    while (XCHG(&l->guard, true));
    if (l->lock) {
        qadd(l->q, tid);
        l->guard = false;
        park();      // blocked
    } else {
        l->lock = true;
        l->guard = false;
    }
}
```

```
void release(LockT *l) {
    while (XCHG(&l->guard, true));
    if (qempty(l->q)) l->lock=false;
    else unpark(qremove(l->q));
    l->guard = false;
}
```

RACE CONDITION

Thread 1

(in lock)

Thread 2

(in unlock)

FINAL correct LOCK

```
typedef struct {  
    bool lock = false;  
    bool guard = false;  
    queue_t q;  
} LockT;
```

setpark() fixes race condition
Park() does not block if unpark()
occured after setpark()

```
void acquire(LockT *l) {  
    while (TAS(&l->guard, true));  
    if (l->lock) {  
        qadd(l->q, tid);  
        setpark(); // notify of plan  
        l->guard = false;  
        park(); // unless unpark()  
    } else {  
        l->lock = true;  
        l->guard = false;  
    }  
}  
  
void release(LockT *l) {  
    while (TAS(&l->guard, true));  
    if (qempty(l->q)) l->lock=false;  
    else unpark(qremove(l->q));  
    l->guard = false;  
}
```

Spin-Waiting vs Blocking

Each approach is better under different circumstances

Uniprocessor

Waiting process is scheduled □ Process holding lock isn't

Waiting process should always relinquish processor

Associate queue of waiters with each lock (as in previous implementation)

Multiprocessor

Waiting process is scheduled □ Process holding lock might be

Spin or block depends on how long, t , before lock is released

Lock released quickly □ Spin-wait

Lock released slowly □ Block

Quick and slow are relative to context-switch cost, C

When to Spin-Wait? When to Block?

If know how long, t , before lock released, can determine optimal behavior

How much CPU time is wasted when spin-waiting?

How much wasted when blocking?

What is the best action when $t < C$?

When $t > C$?

Problem:

Requires knowledge of future; too much overhead to do any special prediction

Two-Phase Waiting

Algorithm: Spin-wait for C then block $\square$ Factor of 2 of optimal

Two cases:

$t < C$: optimal spin-waits for t ; we spin-wait t too

$t > C$: optimal blocks immediately (cost of C);

we pay spin C then block (cost of $2C$);

$2C / C \square$ 2-competitive algorithm

Linux Futex() is an example of a two-phase waiting lock...

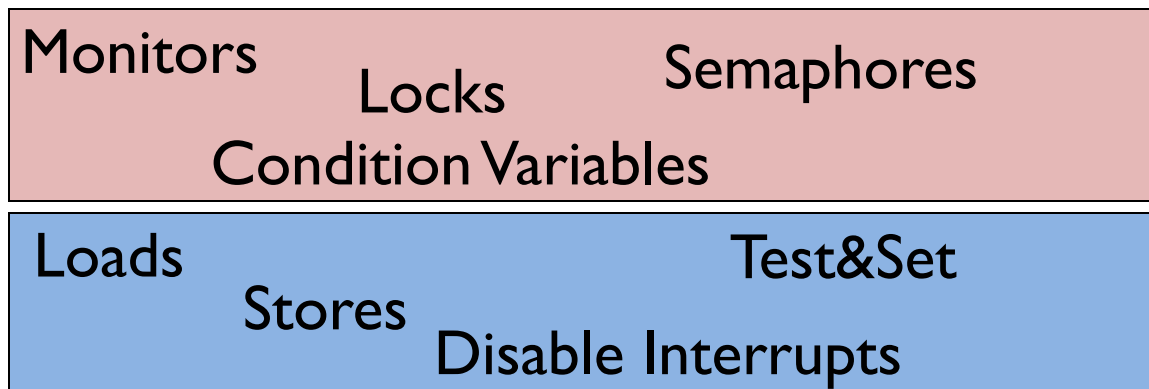
Synchronization

Build higher-level synchronization primitives in OS

Operations that ensure correct ordering of instructions across threads

Use help from hardware

Motivation: Build them once and get them right



CONCURRENCY OBJECTIVES

Mutual exclusion (e.g., A and B don't run at same time)

- solved with *locks*

Ordering (e.g., B runs after A does something)

- solved with *condition variables* and *semaphores*

CONDITION VARIABLES

Condition Variable: queue of waiting threads

B waits for a signal on CV before running

- wait(CV, ...)

A sends signal to CV when time for **B** to run

- signal(CV, ...)

NEXT: CONDITION VARIABLES IN DETAIL

Many examples with CVs